2교시: Plotly & Altair 인터랙티브 차트

> **목표**: Streamlit 내장 차트를 넘어, **플로틀리(Plotly Express)**와 **알테어(Altair)**를 활용하여 인터랙티브하고 시각적으로 풍부한 차트를 구현한다. 데이터프레임 행 선택 이벤트를 활용한 동적 시각화를 경험한다.

---

1. 왜 외부 차트 라이브러리인가?

Streamlit 내장 차트의 한계

3주차에서 `st.line_chart()`, `st.bar_chart()` 등을 사용했습니다. 간편하지만 한계가 있습니다.

기능	내장 차트	Plotly	Altair
코드량	1줄	2~5줄	3~7줄
차트 종류	선, 바, 산점도, 영역	**30종 이상** (파이, 3D, 지도 등)	**다양** (히트맵, 밀도, 스트립 등)
호버 툴팁	기본	**완전 커스터마이징**	**완전 커스터마이징**
색상/스타일	제한적	자유로운 테마/색상	자유로운 인코딩
클릭 이벤트	없음	**on_select 지원**	**on_select 지원**
애니메이션	없음	**프레임 애니메이션**	없음

설치 확인

```
pip install plotly altair
```

> Streamlit을 설치하면 Altair는 이미 포함되어 있습니다. Plotly만 추가 설치하면 됩니다.

> ⚠ **Altair 버전 주의**: `st.altair_chart()`의 `on_select` 기능 등 최신 인터랙션은 **Altair 5.0 이상**에서 안정적으로 작동합니다. 실습 중 오류가 발생하면 아래 명령으로 버전을 확인/업그레이드하세요.
>
> ```
> pip show altair            # 현재 버전 확인
> pip install --upgrade altair  # 최신 버전으로 업그레이드
> ```

---

2. Plotly Express — 빠르고 강력한 시각화

Plotly란?

Plotly는 인터랙티브 차트를 만들어주는 Python 라이브러리입니다. "인터랙티브"란 차트 위에 **마우스를 올리면 값이 표시**되고, **드래그로 확대/축소**하고, **클릭으로 데이터를 선택**할 수 있다는 뜻입니다. 공식 사이트 : https://plotly.com

Plotly에는 두 가지 사용법이 있습니다:

- **plotly.graph_objects** (저수준): 세밀한 제어가 가능하지만 코드가 길어짐
- **plotly.express** (고수준): 한 줄로 차트 생성, 우리는 이것을 사용

Plotly Express는 `px`라는 약자로 가져옵니다. "데이터 → 차트" 변환을 한 줄의 함수 호출로 처리합니다.

Plotly의 기본 사용 패턴

```
import streamlit as st
import plotly.express as px      # 1) 라이브러리 가져오기
import pandas as pd

fig = px.line(df, x='월', y='매출')  # 2) 차트 생성 → fig 객체 반환
#     ↑                              fig = Figure (차트 객체)
#     px.차트종류(데이터, x축, y축)

fig.update_layout(title='제목')   # 3) 스타일 수정 (선택사항)

st.plotly_chart(fig)              # 4) Streamlit에 표시
#  ↑
#  st.plotly_chart()으로 Streamlit 화면에 렌더링
```

> **핵심**: Plotly는 `fig`라는 **차트 객체**를 만들고, 이것을 `st.plotly_chart()`로 Streamlit에 전달하는 2단계 구조입니다. `fig.update_layout()`으로 생성 후에도 스타일을 수정할 수 있습니다.

> 💡 **`use_container_width=True`를 반드시 넣자** `st.plotly_chart(fig)`만 쓰면 차트가 **Plotly 기본 크기(약 700px)**로 고정되어, `st.columns()`나 `layout="wide"` 설정과 상관없이 **너무 작거나 레이아웃을 벗어날 수 있습니다**. `use_container_width=True`를 추가하면 차트가 **현재 위치한 컨테이너(컬럼, 탭 등)의 너비에 자동으로 맞춰집니다**. 이번 실습에서는 모든 예제에 이 옵션을 포함하고 있습니다.
>
> ```
> # ❌ 차트 크기가 고정되어 레이아웃과 맞지 않을 수 있음
> st.plotly_chart(fig)
>
> # ☑ 컨테이너 너비에 맞게 자동 조절
> st.plotly_chart(fig, use_container_width=True)
> ```

2-1. 기본 차트 5종 실습

> 🗀 소스 파일: `source/plotly_basics.py`

```
# plotly_basics.py
import streamlit as st
import plotly.express as px
import pandas as pd
import numpy as np
```

```
st.set_page_config(page_title="Plotly 기초", layout="wide")
st.title('📊 Plotly Express 기본 차트')

# ---- 데이터 생성 ----
np.random.seed(42)
df = pd.DataFrame({
    '월': ['1월','2월','3월','4월','5월','6월','7월','8월','9월','10월','11월','12
월'],
    '매출': np.random.randint(500, 3000, 12),
    '이익': np.random.randint(100, 800, 12),
    '고객수': np.random.randint(50, 300, 12),
    '카테고리': np.random.choice(['온라인', '오프라인'], 12)
})

# ---- 탭으로 차트 종류 분류 ----
tab1, tab2, tab3, tab4, tab5 = st.tabs(['선 그래프', '막대 그래프', '산점도', '파이
차트', '히스토그램'])

with tab1:
    st.subheader('Line Chart')
    fig = px.line(df, x='월', y='매출',
                  title='월별 매출 추이',
                  markers=True,           # 점 표시
                  color_discrete_sequence=['#22d3ee'])  # cyan 색상
    fig.update_layout(
        plot_bgcolor='rgba(0,0,0,0)',     # 배경 투명
        paper_bgcolor='rgba(0,0,0,0)',
        font_color='#f8fafc'
    )
    st.plotly_chart(fig, use_container_width=True)

with tab2:
    st.subheader('Bar Chart')
    fig = px.bar(df, x='월', y='매출',
                 color='카테고리',          # 카테고리별 색상 구분
                 title='월별·채널별 매출',
                 barmode='group',          # 'group' 또는 'stack'
                 color_discrete_map={'온라인': '#22d3ee', '오프라인': '#a855f7'})
    st.plotly_chart(fig, use_container_width=True)

with tab3:
    st.subheader('Scatter Plot')
    fig = px.scatter(df, x='고객수', y='매출',
                     size='이익',           # 버블 크기
                     color='카테고리',
                     hover_name='월',       # 호버 시 이름 표시
                     title='고객수 vs 매출 (버블: 이익)')
    st.plotly_chart(fig, use_container_width=True)

with tab4:
    st.subheader('Pie Chart')
    fig = px.pie(df, names='카테고리', values='매출',
                 title='채널별 매출 비중',
                 color_discrete_map={'온라인': '#22d3ee', '오프라인': '#a855f7'},
```

```
                    hole=0.4)                # 도넛 차트 (가운데 구멍)
    st.plotly_chart(fig, use_container_width=True)

with tab5:
    st.subheader('Histogram')
    fig = px.histogram(df, x='매출',
                       nbins=8,
                       title='매출 분포',
                       color_discrete_sequence=['#deff9a'])
    st.plotly_chart(fig, use_container_width=True)
```

코드 핵심 해설:

```
# color 파라미터 — 색상을 지정하는 3가지 방법

# 1) color_discrete_sequence: 차트 전체에 적용할 색상 목록 (순서대로)
px.line(df, x='월', y='매출', color_discrete_sequence=['#22d3ee'])
#  → 모든 선이 cyan 색상

# 2) color_discrete_map: 특정 카테고리에 특정 색상 매핑 (딕셔너리)
px.bar(df, x='월', y='매출', color='카테고리',
       color_discrete_map={'온라인': '#22d3ee', '오프라인': '#a855f7'})
#  → '온라인'은 cyan, '오프라인'은 purple

# 3) color_continuous_scale: 숫자 값에 따라 그라데이션 색상
px.scatter(df, x='면적', y='인구', color='기온',
           color_continuous_scale='Viridis')
#  → 기온이 낮으면 보라색, 높으면 노란색 (연속 그라데이션)

# use_container_width=True — Streamlit 화면 너비에 맞게 차트 크기 자동 조절
st.plotly_chart(fig, use_container_width=True)
```

2-2. Plotly Express 핵심 함수 정리

함수	차트 종류	주요 파라미터
px.line()	선 그래프	markers, line_dash
px.bar()	막대 그래프	barmode ("group", "stack"), orientation ("h")
px.scatter()	산점도	size, color, hover_name, trendline
px.pie()	파이/도넛	hole (도넛), names, values
px.histogram()	히스토그램	nbins, marginal ("box", "violin")
px.box()	박스 플롯	points ("all", "outliers")
px.sunburst()	선버스트	path (계층), values
px.treemap()	트리맵	path, values, color

2-3. Plotly 스타일 커스터마이징

```
fig.update_layout(
    title_font_size=20,
    xaxis_title='X축 제목',
    yaxis_title='Y축 제목',
    legend_title='범례',
    template='plotly_dark',       # 테마: plotly, plotly_dark, seaborn 등
    hovermode='x unified',       # 호버 모드: x, y, closest, x unified
)

# 축 포맷팅
fig.update_xaxes(tickangle=45)   # X축 라벨 회전
fig.update_yaxes(tickformat=',')  # 천 단위 콤마
```

2-4. 실습: Plotly 이벤트 — 차트 클릭으로 상세 정보 표시

> 🗀 소스 파일: `source/plotly_event.py`

이벤트(Event) 기반 상호작용이란?

지금까지는 Streamlit 위젯(버튼, 슬라이더)으로 사용자 입력을 받았습니다. **차트 이벤트**를 사용하면 **차트 자체가 입력 장치**가 됩니다.

```
[기존 방식]
사용자가 사이드바에서 "서울" 선택 → 서울 데이터만 표시

[이벤트 방식]
사용자가 차트에서 "서울" 점을 클릭 → 서울 상세 정보 표시
```

`st.plotly_chart()`에 `on_select="rerun"` 파라미터를 추가하면, 사용자가 차트에서 **데이터를 클릭/선택**했을 때 스크립트가 재실행되고, 선택 정보를 `event` 변수로 받을 수 있습니다.

```
# plotly_event.py
import streamlit as st
import plotly.express as px
import pandas as pd
import numpy as np

st.set_page_config(page_title="Plotly 이벤트", layout="wide")
st.title('🖱 차트 클릭 → 상세 정보')

# ---- 데이터 ----
np.random.seed(42)
df = pd.DataFrame({
    '도시': ['서울', '부산', '대구', '인천', '광주', '대전', '울산', '세종'],
    '인구(만)': [970, 340, 240, 295, 145, 150, 114, 38],
```

```
    '면적(km²)': [605, 770, 884, 1063, 501, 540, 1062, 465],
    '평균기온(°C)': [12.5, 14.7, 14.1, 12.1, 13.8, 13.0, 14.3, 12.4]
})

# ---- 산점도: 인구 vs 면적 ----
fig = px.scatter(df, x='면적(km²)', y='인구(만)',
                 size='인구(만)', color='평균기온(°C)',
                 hover_name='도시',
                 title='도시별 인구 vs 면적',
                 color_continuous_scale='Viridis')

# on_select="rerun" → 사용자가 점을 클릭하면 Streamlit이 재실행됨
event = st.plotly_chart(fig, on_select="rerun", use_container_width=True)

# ---- 선택된 데이터 표시 ----
if event and event.selection and event.selection.points:
    selected_points = event.selection.points
    st.subheader('선택한 도시 정보')
    for point in selected_points:
        idx = point['point_index']
        city = df.iloc[idx]
        col1, col2, col3, col4 = st.columns(4)
        col1.metric('도시', city['도시'])
        col2.metric('인구', f"{city['인구(만)']}만 명")
        col3.metric('면적', f"{city['면적(km²)']} km²")
        col4.metric('평균기온', f"{city['평균기온(°C)']}°C")
else:
    st.info('💡 차트에서 점을 클릭하면 해당 도시의 상세 정보가 표시됩니다. (Box Select
또는 Lasso Select 도구로 여러 점을 선택할 수도 있습니다)')
```

코드 핵심 해설:

```
# 1) 이벤트 수신 — event 변수에 사용자 선택 정보가 담김
event = st.plotly_chart(fig, on_select="rerun")
#  on_select="rerun" → 사용자가 점을 클릭하면 스크립트가 재실행됨
#  event             → 재실행 시 선택 정보가 담긴 객체가 반환됨

# 2) 선택 정보 확인
event.selection.points    # 사용자가 클릭한 점들의 리스트
# 예: [{'point_index': 0, 'x': 605, 'y': 970}, ...]

# 3) 원본 데이터와 연결
point['point_index']      # 클릭한 점이 DataFrame의 몇 번째 행인지
df.iloc[idx]              # 해당 행의 전체 데이터를 가져옴
```

> **정리**: `on_select="rerun"`을 추가하면 차트가 "클릭 가능"해지고, 클릭한 데이터의 인덱스를 통해 원본 DataFrame에서 상세 정보를 꺼낼 수 있습니다.

---

3. Altair — 선언적 시각화 문법

Altair란?

Altair는 Plotly와 같은 인터랙티브 차트 라이브러리이지만, **문법 철학이 다릅니다**.

	Plotly Express	Altair
접근법	"이 함수를 호출해" (명령형)	"이 데이터를 이렇게 보여줘" (선언형)
코드 예시	`px.line(df, x='날짜', y='매출')`	`Chart(df).mark_line().encode(x='날짜', y='매출')`
비유	요리사에게 "볶음밥 만들어주세요"	레시피를 작성: "재료→인코딩→결과"

선언적(declarative) 이란 "어떻게 그릴지"는 라이브러리에 맡기고, 개발자는 **"무엇을 보여줄지"만 선언**하는 방식입니다. 처음에는 낯설지만, 복잡한 차트를 만들 때 오히려 코드가 깔끔해집니다.

3-1. Altair 기본 문법

```
import altair as alt

chart = alt.Chart(df).mark_타입().encode(
    x='컬럼명',
    y='컬럼명',
    color='컬럼명'
)
```

구성 요소	역할	예시
`alt.Chart(df)`	데이터 지정	DataFrame 전달
`.mark_*()`	시각화 타입	`mark_line()`, `mark_bar()`, `mark_circle()`, `mark_area()`
`.encode()`	데이터 → 시각 속성 매핑	`x`, `y`, `color`, `size`, `tooltip`

> **`.encode()`가 핵심입니다**: "x축에는 날짜를, y축에는 매출을, 색상에는 카테고리를 **인코딩(매핑)** 하겠다"라고 선언하는 것입니다. Altair가 알아서 축 범위, 색상 팔레트, 범례 등을 자동으로 설정합니다.

```
# Plotly와 Altair 비교 — 같은 차트를 두 방식으로 만들기

# Plotly: 함수에 파라미터를 전달
fig = px.line(df, x='날짜', y='매출', color='카테고리')
st.plotly_chart(fig)

# Altair: 체이닝으로 단계별 구성
chart = alt.Chart(df).mark_line().encode(x='날짜', y='매출', color='카테고리')
st.altair_chart(chart)
```

3-2. 기본 차트 실습

> 🗀 소스 파일: `source/altair_basics.py`

```
# altair_basics.py
import streamlit as st
import altair as alt
import pandas as pd
import numpy as np

st.set_page_config(page_title="Altair 기초", layout="wide")
st.title('📊 Altair 기본 차트')

# ---- 데이터 ----
np.random.seed(42)
dates = pd.date_range('2025-01-01', periods=90)
df = pd.DataFrame({
    '날짜': dates,
    '매출': np.cumsum(np.random.randn(90) * 50 + 10),
    '카테고리': np.tile(['A', 'B', 'C'], 30)
})

tab1, tab2, tab3 = st.tabs(['선 그래프', '막대 그래프', '산점도 + 인터랙션'])

with tab1:
    st.subheader('Line Chart')
    chart = alt.Chart(df).mark_line(strokeWidth=2).encode(
        x='날짜:T',              # T = Temporal (시간)
        y='매출:Q',              # Q = Quantitative (수치)
        color='카테고리:N',      # N = Nominal (명목형)
        tooltip=['날짜:T', '매출:Q', '카테고리:N']
    ).properties(
        height=400
    ).interactive()              # 줌/팬 활성화

    st.altair_chart(chart, use_container_width=True)

with tab2:
    st.subheader('Bar Chart')
    bar_data = df.groupby('카테고리')['매출'].mean().reset_index()
    chart = alt.Chart(bar_data).mark_bar(cornerRadiusTopLeft=8,
cornerRadiusTopRight=8).encode(
        x=alt.X('카테고리:N', axis=alt.Axis(labelAngle=0)),
        y='매출:Q',
        color=alt.Color('카테고리:N', legend=None),
        tooltip=['카테고리', '매출']
    ).properties(height=400)

    st.altair_chart(chart, use_container_width=True)

with tab3:
    st.subheader('Scatter + 브러시 선택')
```

```
    # 브러시 선택 영역 정의
    brush = alt.selection_interval()

    scatter = alt.Chart(df).mark_circle(size=60).encode(
        x='날짜:T',
        y='매출:Q',
        color=alt.condition(brush, '카테고리:N', alt.value('lightgray')),
        tooltip=['날짜:T', '매출:Q', '카테고리:N']
    ).add_params(brush).properties(height=400)

    st.altair_chart(scatter, use_container_width=True)
    st.caption('드래그로 영역을 선택하면 해당 점만 색상이 표시됩니다')
```

코드 핵심 해설:

```
# .interactive() — 마우스 줌/팬 활성화
chart = alt.Chart(df).mark_line().encode(...).interactive()
#  마우스 휠로 확대/축소, 드래그로 차트 이동 가능

# .properties() — 차트 크기, 제목 등 속성 설정
chart = alt.Chart(df).mark_line().encode(...).properties(height=400, title='제목')

# alt.X(), alt.Y() — 축을 상세하게 설정할 때 사용
alt.X('카테고리:N', axis=alt.Axis(labelAngle=0))
#  ↑ 축 설정 객체    ↑ X축 라벨을 수평으로 표시 (0도 회전)

# alt.selection_interval() — 드래그로 영역을 선택하는 "브러시" 도구
brush = alt.selection_interval()
#  사용자가 차트 위에서 마우스를 드래그하면 사각형 영역이 선택됨

# alt.condition() — 조건에 따라 다른 시각 속성 적용
color=alt.condition(brush, '카테고리:N', alt.value('lightgray'))
#  "브러시로 선택된 점은 카테고리 색상, 선택 안 된 점은 회색"

# .add_params(brush) — 차트에 인터랙션 파라미터 등록
scatter = alt.Chart(df).mark_circle().encode(...).add_params(brush)
```

3-3. Altair 데이터 타입 약어

Altair는 컬럼 이름 뒤에 `:타입`을 붙여 데이터 타입을 지정합니다.

약어	타입	설명	예시
`:Q`	Quantitative	연속 수치형	매출, 점수, 온도
`:N`	Nominal	명목형 (순서 없음)	카테고리, 이름, 도시
`:O`	Ordinal	순서형	학년 (1,2,3,4), 등급 (상,중,하)
`:T`	Temporal	시간형	날짜, 타임스탬프

3-4. 레이어드 차트 — 차트 위에 차트 겹치기

Altair의 강력한 기능 중 하나는 **여러 차트를 레이어로 합성**하는 것입니다. `+` 연산자로 차트를 겹칠 수 있습니다.

레이어드(Layered) 란 투명한 필름을 여러 장 겹치는 것과 같습니다. 각 레이어에 서로 다른 시각 요소(선, 점, 텍스트 등)를 그리고, 이를 합치면 풍부한 차트가 완성됩니다.

```
Layer 1: 선 그래프 (주가 추이)      ┐
Layer 2: 호버 포인트 (마우스 올리면)  │ + 연산자로
Layer 3: 수직 가이드라인             │ 모두 합침
Layer 4: 주석 이모지 마커            ┘
                                    ↓
                            하나의 완성된 차트
```

> 🗀 소스 파일: `source/altair_layered.py`

```python
# altair_layered.py
import streamlit as st
import altair as alt
import pandas as pd
import numpy as np

st.set_page_config(page_title="Altair 레이어드", layout="wide")
st.title('📊 Altair 레이어드 차트 — 주석 달기')

# ---- 주가 데이터 시뮬레이션 ----
np.random.seed(42)
dates = pd.date_range('2025-01-01', periods=180, freq='D')
prices = 50000 + np.cumsum(np.random.randn(180) * 500)
df = pd.DataFrame({'날짜': dates, '주가': prices})

# ---- 주석(annotation) 데이터 ----
annotations = pd.DataFrame({
    '날짜': pd.to_datetime(['2025-02-14', '2025-04-01', '2025-06-15']),
    '주가': [df.loc[df['날짜'] == '2025-02-14', '주가'].values[0],
             df.loc[df['날짜'] == '2025-04-01', '주가'].values[0],
             df.loc[df['날짜'] == '2025-06-15', '주가'].values[0]],
    '마커': ['⚡', '📊', '💧'],
    '설명': ['실적 발표', '신제품 출시', '최고가 경신']
})

# ---- Layer 1: 선 그래프 (주가 추이) ----
line = alt.Chart(df).mark_line(color='#22d3ee', strokeWidth=2).encode(
    x='날짜:T',
    y=alt.Y('주가:Q', scale=alt.Scale(zero=False)),
    tooltip=['날짜:T', '주가:Q']
)

# ---- Layer 2: 호버 포인트 ----
nearest = alt.selection_point(nearest=True, on='pointerover', fields=['날짜'],
```

```
empty=False)

points = alt.Chart(df).mark_point(size=80, color='#22d3ee').encode(
    x='날짜:T',
    y='주가:Q',
    opacity=alt.condition(nearest, alt.value(1), alt.value(0))
).add_params(nearest)

# ---- Layer 3: 수직 룰 (호버 가이드라인) ----
rules = alt.Chart(df).mark_rule(color='gray', strokeDash=[4,4]).encode(
    x='날짜:T',
    opacity=alt.condition(nearest, alt.value(0.5), alt.value(0))
).add_params(nearest)

# ---- Layer 4: 주석 마커 ----
annotation_marks = alt.Chart(annotations).mark_text(
    size=24, dy=-20
).encode(
    x='날짜:T',
    y='주가:Q',
    text='마커:N',
    tooltip=['설명:N', '날짜:T', '주가:Q']
)

# ---- 모든 레이어 합성 ----
chart = (line + points + rules + annotation_marks).properties(
    height=450,
    title='시뮬레이션 주가 차트 (주석 포함)'
).interactive()

st.altair_chart(chart, use_container_width=True)
st.caption('마우스를 올리면 포인트와 가이드라인이 표시됩니다. 이모지 마커를 호버하면 이벤트 설명이 나타납니다.')
```

레이어 합성 연산자:

연산자	의미	사용
`+`	레이어 (겹치기)	같은 공간에 여러 차트 겹침
`|`	가로 배치	차트를 옆으로 나란히
`&`	세로 배치	차트를 위아래로 쌓기

---

4. 데이터프레임 행 선택 이벤트

3주차에서 `st.dataframe()`을 **데이터를 보여주는 용도**로 사용했습니다. 여기에 `on_select` 파라미터를 추가하면, 테이블의 **행을 클릭하여 선택**하고 선택된 행의 데이터를 차트에 반영하는 **"선택 → 분석" 워크플로우**를 만들 수 있습니다.

```
┌──────────────────────┐
│ 제품 | 매출 | 성장률 |  ← 테이블에서 행을 클릭하여 선택
|──────|──────|───────|
|▶노트북| 150  | 12.3% |  ← ▶ 선택됨
| 태블릿|  80  |  5.1% |
|▶스마트| 180  | 22.0% |  ← ▶ 선택됨
└──────────────────────┘
        ↓ 선택된 행의 데이터가 차트에 반영

  ┌──────────┐  ┌──────────┐
  | 매출 비교  |  | 성장률 비교 |
  | ██████ 150 |  | ███   12% |
  | ████████180|  | ██████ 22% |
  └──────────┘  └──────────┘
```

📁 소스 파일: `source/dataframe_selection.py`

```python
# dataframe_selection.py
import streamlit as st
import plotly.express as px
import pandas as pd
import numpy as np

st.set_page_config(page_title="행 선택", layout="wide")
st.title('🔍 데이터프레임 행 선택 → 차트 비교')

# ---- 데이터 ----
np.random.seed(42)
products = ['노트북', '태블릿', '스마트폰', '이어폰', '스마트워치', '키보드', '마우스', '모니터']
df = pd.DataFrame({
    '제품': products,
    '매출(억)': np.random.randint(10, 200, len(products)),
    '성장률(%)': np.random.uniform(-5, 25, len(products)).round(1),
    '월별추이': [np.random.randint(5, 30, 12).tolist() for _ in products]
})

st.subheader('제품 목록 (행을 클릭하여 선택)')

# ---- 행 선택 가능한 데이터프레임 ----
event = st.dataframe(
    df[['제품', '매출(억)', '성장률(%)']],
    on_select="rerun",
    selection_mode="multi-row",
    use_container_width=True,
    column_config={
        '성장률(%)': st.column_config.ProgressColumn(
            min_value=-10, max_value=30, format='%.1f%%'
        )
    }
)
```

```
# ---- 선택된 행으로 비교 차트 ----
selected_rows = event.selection.rows
if selected_rows:
    selected_df = df.iloc[selected_rows]
    st.subheader(f'선택한 {len(selected_rows)}개 제품 비교')

    col1, col2 = st.columns(2)
    with col1:
        fig = px.bar(selected_df, x='제품', y='매출(억)',
                     color='제품', title='매출 비교')
        st.plotly_chart(fig, use_container_width=True)

    with col2:
        fig = px.bar(selected_df, x='제품', y='성장률(%)',
                     color='제품', title='성장률 비교')
        st.plotly_chart(fig, use_container_width=True)
else:
    st.info('💡 위 테이블에서 행을 클릭하면 비교 차트가 표시됩니다.')
```

st.column_config — 컬럼 커스터마이징

`st.dataframe()`의 `column_config`로 컬럼 표시 방식을 변경할 수 있습니다.

설정	설명
`st.column_config.ProgressColumn()`	프로그레스 바로 표시
`st.column_config.LineChartColumn()`	셀 안에 스파크라인 차트
`st.column_config.BarChartColumn()`	셀 안에 바 차트
`st.column_config.ImageColumn()`	이미지 URL을 이미지로 표시
`st.column_config.LinkColumn()`	URL을 클릭 가능한 링크로 표시

> 💡 **column_config를 쓰면 "표"가 "대시보드"가 된다** 기본 `st.dataframe()`은 숫자와 텍스트만 나열합니다. 하지만 `column_config`를 적용하면 **셀 안에 차트, 프로그레스 바, 이미지**가 들어가면서 단순한 표가 시각적인 대시보드로 변합니다. 기말 프로젝트에서 대시보드를 구현할 때 가장 유용한 기능이니, 다양한 Column 타입을 직접 바꿔보며 익혀두세요.

```
# 예시: 여러 column_config를 조합하면 풍부한 테이블이 완성됨
st.dataframe(
    df,
    column_config={
        '성장률': st.column_config.ProgressColumn(min_value=-10, max_value=30,
format='%.1f%%'),
        '월별추이': st.column_config.LineChartColumn(y_min=0, y_max=30),
        '로고': st.column_config.ImageColumn(width='small'),
    },
    use_container_width=True
)
```

5. Plotly vs Altair — 언제 무엇을 쓸까? (Tip)

기준	Plotly Express	Altair
문법 스타일	함수 호출형 (`px.line(df, ...)`)	체이닝형 (`Chart(df).mark_line().encode(...)`)
학습 곡선	낮음 (직관적)	중간 (문법 이해 필요)
차트 종류	매우 다양 (3D, 지도, 금융 등)	통계/분석 중심
인터랙션	줌, 팬, 호버 기본 내장	브러시 선택, 조건부 색상 강력
레이어 합성	`add_trace()`로 가능	`+` 연산자로 직관적
추천 상황	빠르게 다양한 차트 그리기	데이터 탐색, 통계 시각화

> **결론**: 프로젝트에서는 **Plotly Express**를 기본으로 사용하고, **통계적 탐색이나 브러시 선택이 필요한 경우 Altair**를 활용하세요. 두 라이브러리를 같은 앱에서 섞어 써도 됩니다.

6. 실습 팁 모음

💡 내장 데이터셋으로 빠르게 연습하기

Plotly Express에는 연습용 내장 데이터셋이 포함되어 있어, **CSV 파일을 준비하지 않아도** 바로 실습할 수 있습니다.

```
import plotly.express as px

# 내장 데이터셋 목록
gapminder = px.data.gapminder()   # 국가별 기대수명/GDP/인구 (1952~2007)
iris = px.data.iris()             # 붓꽃 꽃잎/꽃받침 측정값
tips = px.data.tips()             # 레스토랑 팁 데이터
stocks = px.data.stocks()         # 주가 시계열

# 예시: 한 줄로 인터랙티브 차트 완성
fig = px.scatter(iris, x='sepal_width', y='sepal_length',
                 color='species', size='petal_length',
                 title='붓꽃 데이터 탐색')
st.plotly_chart(fig, use_container_width=True)
```

> 수업 중 빠르게 차트를 시연하거나, 학생들이 개인 실습할 때 데이터 파일 없이 바로 시작할 수 있어 편리합니다.

🔗 1교시 캐싱과의 연결

2교시에서 만드는 Plotly/Altair 차트는 데이터를 불러오고 가공하는 과정이 포함됩니다. 위젯을 조작할 때마다 데이터가 매번 다시 로드되면 차트 렌더링이 느려집니다.

```python
# 1교시에서 배운 캐싱을 데이터 로딩에 적용!
@st.cache_data
def load_data():
    return px.data.gapminder()  # 또는 pd.read_csv('대용량파일.csv')

df = load_data()  # 첫 실행 이후에는 캐시에서 즉시 반환

# 이후 위젯 조작 시 데이터 로딩은 건너뛰고, 차트만 다시 그림
year = st.slider('연도', 1952, 2007, 2007, step=5)
filtered = df[df['year'] == year]
fig = px.scatter(filtered, x='gdpPercap', y='lifeExp',
                 size='pop', color='continent', hover_name='country')
st.plotly_chart(fig, use_container_width=True)
```

> **기억하세요**: 데이터 로딩에는 `@st.cache_data`, 차트를 그리는 코드 자체는 위젯 값에 따라 달라져야 하므로 캐싱하지 않습니다.

2교시 마무리 체크리스트

- ☐ Plotly Express로 선, 막대, 산점도, 파이, 히스토그램 차트를 만들 수 있다
- ☐ `fig.update_layout()`으로 차트 스타일을 커스터마이징할 수 있다
- ☐ `st.plotly_chart(on_select="rerun")`로 차트 클릭 이벤트를 처리할 수 있다
- ☐ Altair의 `Chart().mark_*().encode()` 패턴으로 차트를 만들 수 있다
- ☐ Altair 데이터 타입 약어 (`:Q`, `:N`, `:O`, `:T`)의 의미를 알고 있다
- ☐ Altair `+` 연산자로 레이어드 차트를 합성할 수 있다
- ☐ `st.dataframe(on_select="rerun")`로 행 선택 이벤트를 처리할 수 있다
- ☐ `st.column_config`로 데이터프레임 컬럼 표시 방식을 커스터마이징할 수 있다